Personalized Mental Health Assistant

A project report submitted
in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

BY

Name- Mahvish Ruhi , Roll no – 23000121025, Reg no - 212300100110007
Name : Sayan Bhattacharya, Roll : 23000122060 Reg. No: 222300120128
Name: Prantik Ghosh, Roll: 23000122059, Reg. No.: 222300120120
Name: Sanchita Kar, Roll:23000121019, Reg. No. : 212300100110018
Name: Arghyakamal Ghosh, Roll:23000121026, Reg. No. : 212300100110009

Pursued in
Department of Computer Science and Engineering

CAMELLIA INSTITUTE OF TECHNOLOGY
Madhyamgram, Kolkata, West Bengal 700128
To
MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGYKOLKATA, WEST BENGAL

January, 2025

CERTIFICATE

This is to certify that this project report titled Personalized Mental Health Assistant submitted in partial fulfillment of requirements of award of the degree Bachelor of Technology in Computer Science and Engineering is a bonafide record carried out by,

Name- Mahvish Ruhi , Roll no – 23000121025, Reg no - 212300100110007
Name : Sayan Bhattacharya, Roll : 23000122060 Reg. No: 222300120128
Name: Prantik Ghosh, Roll: 23000122059, Reg. No.: 222300120120
Name: Sanchita Kar, Roll:23000121019, Reg. No. : 212300100110018
Name: Arghyakamal Ghosh, Roll:23000121026, Reg. No. : 212300100110009

Under my Guidance and Supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree or diploma.

Date: .....

_____

_____
Head of the Department
Dept. of CSE
CAMELLIA INSTITUTE OF TECHNOLOGY

DECLARATION

We hereby declare that this project report titled Personalized Mental Health Assistant is our original work carried out as under graduate student in Camellia Institute of Technology.

This report submitted in partial fulfillment of the degree of B.Tech. (in Computer Science and Engineering) is a record of original work carried out by me under the supervision of Jishno Das and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Student's Name	Signature	Date
Mahvish Ruhi		
.....	.....	.....
Sayan Bhattacharya		
.....	.....	.....
Prantik Ghosh		
.....	.....	.....
Sanchita Kar		
.....	.....	.....
Arghyakamal Ghosh		
.....	.....	.....

ABSTRACT

Mental health challenges are prevalent globally, impacting individuals of all ages and backgrounds. Access to timely and effective mental healthcare remains a significant barrier for many, often hindered by stigma, cost, and limited availability of qualified professionals. This report explores the potential of a Personalized Mental Health Assistant (PMHA), an AI-powered system designed to provide personalized mental health support.

The PMHA leverages advanced machine learning algorithms to analyze user data, including getting information through some questions and suggesting exercise and meditation to help them to handle their mental disorder. By identifying patterns and trends, the system can proactively predict potential mental health concerns and provide tailored interventions. These interventions may include personalized mindfulness exercises, cognitive behavioral therapy techniques.

The PMHA aims to enhance accessibility to mental healthcare by providing 24/7 support, reducing the stigma associated with seeking professional help, and offering affordable and convenient options. By combining cutting-edge technology with personalized care, the PMHA has the potential to revolutionize mental healthcare delivery and improve the mental well-being of individuals worldwide.

Introduction

Mental health challenges have become increasingly prevalent in today's society, impacting individuals of all ages and backgrounds. However, accessing timely and effective mental healthcare can be challenging due to various factors, including stigma, cost, and limited availability of qualified professionals. This report focuses on the development on Personalized Mental Health Assistant (PMHA), an AI-powered system designed to provide accessible and personalized mental health support.

The primary objective of this work is to create an AI-powered system that can proactively identify potential mental health concerns in individuals and offer tailored interventions. Existing research in this area has explored the use of AI for mental health support, including chatbots and AI-powered screening tools. However, many of these approaches lack personalization and fail to provide proactive, preventative care.

The innovative aspect of this work lies in its emphasis on personalized interventions. The PMHA utilizes advanced machine learning algorithms to analyze a wide range of user data, including getting information through some questions and suggesting exercise and meditation to help them to handle their mental disorder.Based on these insights, the system provides tailored recommendations and interventions, such as personalized mindfulness exercises, cognitive behavioral therapy techniques, and access to relevant resources.

The remainder of this report is organized as follows. Chapter 2 provides a brief overview of the background and related work in the field of AI-powered mental health support. Chapter 3 details the methodology employed in the development of the PMHA, including data collection, feature engineering, and model development. Chapter 4 presents the results of our evaluation, demonstrating the effectiveness of the PMHA in identifying potential mental health concerns and providing personalized support. Finally, Chapter 5 discusses the limitations of the current system and outlines future directions for research and development.

1. Relevant Theoretical Aspects

This section of your report should lay the groundwork for the technical and conceptual foundations of your Personalized Mental Health Assistant (PMHA). Here's a breakdown of key theoretical aspects to include:

a) Artificial Intelligence (AI) and Machine Learning (ML)

- **Supervised Learning:**

- **Classification:** Algorithms like Support Vector Machines (SVM), Random Forests, and Logistic Regression can be used to classify users into different risk categories for mental health conditions based on their data.

- **Regression:** Algorithms like Linear Regression and Decision Trees can be used to predict continuous variables related to mental well-being, such as stress levels or anxiety scores.

- **Unsupervised Learning:**

  - **Clustering:** Algorithms like k-means can group users with similar characteristics (e.g., demographics, lifestyle, symptom patterns) to personalize interventions and identify potential subgroups within the population.

- **Deep Learning:**

  - **Neural Networks:** Deep Neural Networks (DNNs), including Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), can analyze complex patterns in time-series data (e.g., sleep patterns, activity levels) and identify subtle indicators of mental health changes.

  - **Natural Language Processing (NLP):** Techniques like sentiment analysis, topic modeling, and chatbots can be used to analyze user text input (e.g., journal entries, chat messages) to understand their emotional state and identify potential concerns.

b) Mental Health

- **Cognitive Behavioral Therapy (CBT):**

  - Discuss the core principles of CBT, such as identifying and modifying negative thought patterns and behaviors.
  - Explore how CBT techniques (e.g., cognitive restructuring, relaxation exercises) can be integrated into the PMHA's intervention strategies.

- **Mindfulness and Meditation:**

  - Explain the principles and techniques of mindfulness and meditation, including guided meditations, breathing exercises, and body scan techniques.
  - Discuss how the PMHA can provide access to guided mindfulness exercises and resources.

- **Affective Computing:**

- Discuss the study and analysis of human emotions and their expression.
- Explore how the PMHA can utilize affective computing techniques to recognize and interpret user emotions from various data sources (e.g., facial expressions, voice tone, text sentiment).

c) Ethical Considerations

- **Data Privacy and Security:**
  - Discuss the importance of data privacy and security in the context of mental health.
  - Explain data anonymization techniques, secure data storage methods, and compliance with relevant regulations (e.g., GDPR, HIPAA).

- **Algorithmic Bias:**
  - Acknowledge the potential for bias in AI algorithms, such as biases related to race, gender, and socioeconomic status.
  - Discuss strategies for mitigating bias in data collection, model development, and decision-making.

- **User Trust and Transparency:**
  - Discuss the importance of building user trust and ensuring transparency in the PMHA system.
  - Explain how to provide clear information about data usage, system functionality, and the limitations of the system.

2. Description of Methods

This section will outline the specific methodologies employed in the development and evaluation of the PMHA system.

a) Data Collection

- **User Data:**
  - **Surveys:** Develop and administer surveys to collect demographic information, mental health history (e.g., past diagnoses, medication use), current symptoms (e.g., anxiety levels, depression scores), and lifestyle factors (e.g., sleep quality, exercise habits).

b) Data Preprocessing

- **Data Cleaning:**
  - **Handle Missing Values:** Impute missing data using appropriate methods (e.g., mean imputation, k-nearest neighbors).

- **Handle Outliers:** Identify and address outliers in the data (e.g., by removing them or using robust statistical methods).
- **Data Consistency Checks:** Ensure data consistency and accuracy across different sources.

- **Data Transformation:**
  - **Normalization:** Scale data to a common range (e.g., between 0 and 1) to improve model performance.
  - **Feature Engineering:**
    - Create new features from existing data (e.g., calculate sleep quality metrics, derive activity levels, create time-series features).
    - Extract relevant features from text data using NLP techniques (e.g., sentiment scores, topic keywords).
- **Data Splitting:**
  - Divide the dataset into training, validation, and test sets for model development and evaluation

c) Model Development

- **Model Selection:**
  - Choose appropriate machine learning models based on the specific task and data characteristics:
    - **Classification:** Support Vector Machines (SVM), Random Forests, Logistic Regression, Deep Neural Networks (DNNs) for predicting the presence or absence of mental health conditions.
    - **Regression:** Linear Regression, Decision Trees, and DNNs for predicting continuous variables (e.g., stress levels, anxiety scores).
    - **Clustering:** K-means, hierarchical clustering for identifying user groups with similar characteristics.
    - **Time Series Analysis:** Recurrent Neural Networks (RNNs) for analyzing time-series data (e.g., sleep patterns, activity levels) to identify trends and predict future outcomes.
- **Model Training:**
  - Train selected models on the training dataset.
  - Use appropriate optimization algorithms (e.g., stochastic gradient descent) to minimize the model's loss function.
- **Hyperparameter Tuning:**

- Tune hyperparameters of the selected models to optimize performance (e.g.,learning rate, number of hidden units, regularization strength).
- Use techniques like grid search, random search, or Bayesian optimization.

d) Intervention Recommendation

- **Develop algorithms to generate personalized intervention recommendations based on user data and model predictions:**
  - **Rule-based systems:** Create rules based on user characteristics and risk factors.
  - **Decision trees:** Use decision trees to guide the recommendation process.
  - **Reinforcement learning:** Train agents to learn optimal intervention strategies based on user feedback.
- **Integrate with external resources:**
  - Provide access to relevant resources (e.g., mindfulness apps, therapy databases, support groups) based on user needs.
  - Develop a library of personalized mindfulness exercises, relaxation techniques, and cognitive behavioral therapy (CBT) exercises.

e) Evaluation

- **Model Evaluation:**
  - Evaluate model performance using appropriate metrics:
    - **Classification:** Accuracy, precision, recall, F1-score, AUC-ROC.
    - **Regression:** Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared.
  - Use cross-validation techniques to assess model generalization performance.
- **User Studies:**
  - Conduct user studies to assess the usability, acceptability, and effectiveness of the PMHA system.
  - Collect user feedback on the system's interface, functionality, and the helpfulness of the interventions.

3. Hardware/Software Used

This section will outline the specific hardware and software tools utilized in the development and deployment of your PMHA system.

- **Software**

  - **Programming Languages:**
    - **Backend**: Python with Django

      - Django handles server-side logic, API endpoints, and SQLite3 as the database.

      - Django REST Framework (DRF) is used for building APIs.

    - **Frontend**: React.js with Tailwind CSS

      - React.js creates an interactive user interface.

      - Tailwind CSS provides utility-first styling for modern, responsive designs.

Machine Learning Frameworks:

1. Pre-trained Language Model

- **Model**: microsoft/DialoGPT-medium (a fine-tuned version of GPT-2 for conversational tasks).
- **Framework**: Hugging Face Transformers.
- **Key Features**:
  - The model is pre-trained on large-scale conversational data.
  - It leverages transformer architecture to generate coherent and contextually relevant responses.

2. Backend Infrastructure

- **Framework**: Django with Django REST Framework (DRF).
- **Functionality**:
  - Manages API endpoints for handling user interactions.
  - Hosts the AIChatView API endpoint (ai-chat/) that connects the frontend and AI model.
  - Encodes and decodes user input and AI responses using the pre-trained model.

3. Frontend

- **Technology**: Reactjs, Tailwind CSS (Vanilla JS for simplicity in this example).
- **Key Features**:
  - Provides a user interface for users to interact with the AI assistant.
  - Handles API requests to the backend for sending messages and receiving responses.

4. Communication Layer

- **Protocol**: HTTP with JSON for exchanging messages between the frontend and backend.

- **Endpoint**: /ai-chat/ (POST request).
- **Frontend-Backend Workflow**:
  - User inputs a message in the frontend.
  - The message is sent as a POST request to the backend.
  - Backend processes the message using the AI model and returns a response.
  - Frontend displays the response to the user.

- **Data Storage:-**

a. User Information

Data: Personal details such as name, email, phone number, age, address, occupation, etc.

Storage: Relational database using Django's ORM.

Purpose: Authenticate and identify users.

b. Conversation Logs

Data: Messages exchanged between users and the AI assistant.

- user_message: User input.
- ai_response: AI's generated response.
- Timestamps for when messages were created.

Storage: Relational database using Django's ORM.

Purpose: Analyze user interactions, improve AI responses, and maintain session continuity.

c. AI Model Data

Data: Pre-trained model weights and tokenizer vocabulary files.

Storage: Local file system or a cloud storage service like AWS S3.

Purpose: Load and run the conversational model.

d. Fine-Tuning Datasets

Data: Domain-specific datasets (e.g., EmpatheticDialogues) used for fine-tuning the AI model.

Storage: Local file system or cloud storage.

Purpose: Train and improve the AI assistant.

b) Hardware

- **Computing Resources:**

- **Cloud Computing Instances:** Virtual machines (VMs) or cloud-based servers with sufficient CPU, RAM, and GPU resources for model training and deployment.
- **High-Performance Computing (HPC) Clusters:** For computationally intensive tasks such as training large deep learning models.

- **Development Machines:**
  - Local workstations or laptops with sufficient processing power and memory for development and testing.

4. Description of the Components of the System & Their Interaction

This section will provide a detailed overview of the key components of the PMHA system and how they interact with each other.

a) User Interface (UI)

- **User Registration and Onboarding:**
  - A user-friendly interface for new users to register and provide basic information (e.g., age, gender, contact information).
  - Onboarding process to guide users through the system's features and functionalities.

- **Data Input and Monitoring**
  - Provide what type of Mental disorder Users have and suggest Exercise and meditation based on their Mental disorder.

- **Intervention Delivery:**
  - Display personalized intervention recommendations in a clear and concise format.
  - Provide access to a library of resources (e.g., mindfulness exercises, relaxation techniques, CBT techniques) through the interface.
  - Enable users to schedule and track their progress on recommended interventions.

- **Feedback Mechanisms:**
  - Allow users to provide feedback on the system's functionality, the helpfulness of interventions, and any issues they encounter.

- Collect user feedback to improve the system's performance and user experience.

b) Data Acquisition and Processing

- **Data Collection Modules:**
  - Modules to collect data from various sources (e.g., user input, wearable devices, APIs).
  - Ensure data is collected securely and with user consent.
- **Data Preprocessing Module:**
  - Clean and preprocess the collected data:
    - Handle missing values, outliers, and inconsistencies.
    - Normalize and standardize data features.
    - Extract relevant features from raw data (e.g., sentiment analysis, time-series feature extraction).
- **Data Storage Module:**
  - Store collected data securely and efficiently in a database (e.g., cloud database, local database).
  - Implement data security measures to protect user privacy.

c) Machine Learning Model

- **Model Training and Prediction:**
  - Train and deploy machine learning models to:
    - Predict the risk of mental health issues.
    - Identify patterns and trends in user data.
    - Personalize intervention recommendations.
  - Utilize appropriate models based on the specific task (e.g., classification, regression, time-series analysis).

d) Intervention Recommendation Engine

- **Recommendation Algorithms:**
  - Implement algorithms to generate personalized intervention recommendations based on user data, model predictions, and user preferences.
  - Consider factors like user risk level, current symptoms, and preferred intervention methods.
- **Resource Integration:**
  - Integrate with external resources (e.g., mindfulness apps, therapy databases, support groups) to provide a comprehensive range of interventions.

- Develop a library of personalized mindfulness exercises, relaxation techniques, and CBT exercises.

e) User Feedback and Data Storage

- **Collect and Analyze User Feedback:**
  - Gather user feedback through surveys, in-app feedback mechanisms, and user support channels.
  - Analyze user feedback to identify areas for improvement and make necessary adjustments to the system.
- **Data Storage and Management:**
  - Store user feedback data for future analysis and system improvement.
  - Ensure the security and privacy of user feedback data.

System Interactions

- Users interact with the UI to input data, access interventions, and provide feedback.
- The UI collects user data and sends it to the data acquisition and processing module.
- The data processing module cleans, transforms, and stores the data.
- The machine learning model analyzes the processed data to generate predictions and insights.
- The intervention recommendation engine uses model predictions and user data to generate personalized interventions.
- The UI displays the recommendations to the user and provides access to relevant resources.
- User feedback is collected and used to improve the system's performance and user experience.

1. System Design

This section delves into the core design aspects of the Personalized Mental Health Assistant (PMHA) system. It will incorporate various diagrams and models to illustrate the system's structure, functionality, and data flow.

a) Design

- **High-Level Architecture:**
    - A visual representation of the overall system architecture, outlining the major components (user interface, data acquisition, data processing, machine learning model, intervention recommendation engine, and data storage) and their interactions.
- **Data Flow:**
    - A description of how data flows through the system, from data collection to intervention delivery and feedback loops.

b) Algorithms

- **Data Preprocessing Algorithms:**
    - Algorithms for handling missing data (e.g., imputation methods), outlier detection, and data normalization.
    - Feature engineering algorithms for creating new features from raw data (e.g., time-series feature extraction, sentiment analysis).
- **Machine Learning Algorithms:**
    - Detailed descriptions of the machine learning models used (e.g., SVM, Random Forest, RNNs, CNNs) and their hyperparameter tuning strategies.
- **Intervention Recommendation Algorithms:**
    - Algorithms for generating personalized recommendations based on user data, model predictions, and user preferences.
    - This could include rule-based systems, decision tree-based approaches, or reinforcement learning algorithms.

c) Functional Flowchart

- A visual representation of the system's workflow, showing the sequence of steps involved in data acquisition, processing, analysis, and intervention delivery.
- Illustrates the interactions between different components of the system.

d) Entity-Relationship Diagram (ERD)

- A visual representation of the relationships between different data entities within the system.
- Depicts entities such as "User," "Data Point," "Intervention," "Recommendation," and their attributes and relationships.

e) Data Flow Diagram (DFD)

- A graphical representation of the flow of data within the system.
- Shows how data is collected, processed, stored, and used to generate interventions.

f) Class Diagram

- A UML diagram that illustrates the classes and objects within the system.
- Defines the attributes and methods of each class and the relationships between classes (e.g., inheritance, association).

g) Use Case Diagram

- A UML diagram that depicts the interactions between users and the system.
- Identifies the different user roles (e.g., user, administrator) and the functionalities they can perform (e.g., register, input data, view recommendations, provide feedback).

Backend Development

The backend of the “MentalHealth” project is developed using Django, a Python-based web framework, which ensures rapid development and clean design. The Django REST Framework (DRF) is used to create APIs that handle user registration, login, responses, and disorder recommendations. JWT (JSON Web Tokens) authentication is implemented to secure user sessions and API requests.

Key Features:

- **User Authentication**: Custom user model with JWT-based login and registration.
- **Response Handling**: API endpoints to submit and retrieve responses.

- **Recommendation Engine**: Uses user responses to provide tailored recommendations for mental health improvement.

Example Code Snippets:

User Registration View:

```
class RegisterView(generics.CreateAPIView):
    queryset = RegisterUser.objects.all()
    serializer_class = RegisterSerializer
```

User Login View:

```
class LoginView(generics.GenericAPIView):
    serializer_class = LoginSerializer
    def post(self, request, *args, **kwargs):
        email = request.data.get('email')
        password = request.data.get('password')
        user = authenticate(request, email=email, password=password)
        if user is not None:
            refresh = RefreshToken.for_user(user)
            return Response({
                'refresh': str(refresh),
                'access': str(refresh.access_token),
            })
        return Response({"message": "Invalid credentials"}, status=401)
```

Database Design and Management

The project uses SQLite as the database system for development purposes. The database schema includes models for users, questions, responses, disorders, and their associations.

Key Models:

1. **RegisterUser**: Custom user model for authentication.
2. **Questions**: Stores questions for mental health assessments.
3. **Response**: Links user responses to specific questions.
4. **Disorder**: Represents mental health disorders and their associated recommendations.
5. **DisorderSave**: Links disorders to specific questions.

Example Code Snippets:

RegisterUser Model:

```
class RegisterUser(AbstractBaseUser, PermissionsMixin):
    name = models.CharField(max_length=50)
    email = models.EmailField(unique=True)
    phone_number = models.CharField(max_length=20)
```

```
    dob = models.DateField()
    age = models.IntegerField()
    address = models.TextField(max_length=100)
    occupation = models.CharField(max_length=50)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=True)

    objects = RegisterUserManager()

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ['name', 'phone_number', 'dob', 'age', 'address', 'occupation']
```

Response Model:

```
class Response(models.Model):
    question = models.ForeignKey(Questions, on_delete=models.CASCADE)
    response = models.CharField(max_length=3, choices=response_choices)
    user = models.ForeignKey(RegisterUser, on_delete=models.CASCADE)

    def __str__(self):
        return self.response
```

Database Operations

The database operations are primarily handled through Django's ORM, ensuring secure and efficient data manipulation.

Key Operations:

1. **User Creation**:

   ```
   user = RegisterUser.objects.create_user(
       name=validated_data['name'],
       email=validated_data['email'],
       password=validated_data['password']
   )
   ```

2. **Fetching Questions**:

   ```
   questions = Questions.objects.all()
   ```

3. **Storing Responses**:

   ```
   response = Response.objects.create(user=request.user, question=question,
   response="Yes")
   ```

4. **Recommendation Queries**:

```
disorder_id =
DisorderSave.objects.filter(question_id__in=question_ids).values_list('disorder_id',
flat=True)
disorders = Disorder.objects.filter(id__in=disorder_id).distinct()
```

Functional Modules (Backend-Database Integration)

Modules:

1. **User Authentication**: Handles registration and login with JWT.
2. **Questionnaire Module**: Fetches questions for mental health assessments.
3. **Response Handling**: Submits individual or bulk responses.
4. **Recommendation Engine**: Provides tailored recommendations based on user responses.

Example Integration:

Disorder Recommendation View:

```
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def Disorder_recommendation(request):
    user_responses = Response.objects.filter(user=request.user, response="Yes")
    question_ids = user_responses.values_list('question_id', flat=True)
    disorder_ids =
DisorderSave.objects.filter(question_id__in=question_ids).values_list('disorder_id',
flat=True)
    disorders = Disorder.objects.filter(id__in=disorder_ids).distinct()
    serializer = DisorderSerializer(disorders, many=True)
    return Response({"disorders": serializer.data}, status=status.HTTP_200_OK)
```

Testing and Debugging

Testing Approaches:

1. **Unit Testing**: Ensured the correctness of individual views and serializers.
2. **Integration Testing**: Verified end-to-end functionality of APIs and database interactions.
3. **Manual Testing**: Tested API endpoints using tools like Postman.

Debugging Techniques:

- Utilized Django’s debugging tools and error logs.
- Added print statements for critical queries (later replaced with logging).

Additional Notes

Security Considerations:

- JWT tokens are used for secure authentication.

- Sensitive information like SECRET_KEY is managed through environment variables.

Scalability:

- Though SQLite is used for development, the project is designed to be compatible with more robust databases like PostgreSQL.

Future Enhancements:

- Add machine learning models for more precise disorder predictions.
- Enhance bulk operations with transaction management.
- Optimize queries for large datasets.

Logging Example:

```
import logging
logger = logging.getLogger(__name__)
logger.info("Disorder recommendation process started")
```

AI Chatbot Development

The **AI Chatbot** for the “MentalHealth” project enhances user interaction by providing empathetic and relevant responses to mental health queries. It leverages state-of-the-art natural language processing (NLP) models to ensure meaningful conversations and user engagement.

Backend Development for AI Chatbot

The chatbot backend is built using **Hugging Face Transformers** and integrated into the existing Django application. The AI model is accessed via a REST API endpoint that receives user messages, processes them using the AI model, and returns appropriate responses.

Key Features:

- Uses pre-trained **DialoGPT** for conversational AI.
- Provides a REST API endpoint for frontend integration.
- Generates responses dynamically based on user input.

Code Snippets:

AI Assistant Class:

```
# ai_assistant.py
from transformers import AutoModelForCausalLM, AutoTokenizer

class AIAssistant:
    def __init__(self, model_name="microsoft/DialoGPT-medium"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name)

    def get_response(self, user_input, max_length=100):
```

```
        inputs = self.tokenizer.encode(user_input, return_tensors="pt")
        outputs = self.model.generate(inputs, max_length=max_length,
pad_token_id=self.tokenizer.eos_token_id)
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True)
```

AI Chat View:

```
# views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from .ai_assistant import AIAssistant

class AIChatView(APIView):
    def post(self, request, *args, **kwargs):
        user_message = request.data.get("message", "")
        if not user_message:
            return Response({"error": "Message is required"}, status=400)

        assistant = AIAssistant()
        ai_response = assistant.get_response(user_message)

        return Response({"user_message": user_message, "ai_response": ai_response})
```

API Endpoint Routing:

```
# urls.py
from django.urls import path
from .views import AIChatView

urlpatterns = [
    path("ai-chat/", AIChatView.as_view(), name="ai_chat"),
]
```

Database Design and Management

The chatbot does not require additional database tables for its operation. However, conversations can be logged for future reference or analysis. A potential table for conversation logging could include:

Example Table Design:

Table: ConversationLog | Field Name | Data Type | Description | |————-|————-|————————| | id | AutoField | Primary key | | user_id | ForeignKey | References RegisterUser | | message | TextField | User’s message | | response | TextField | AI’s response | | timestamp | DateTime | Time of interaction |

Functional Modules (Backend Integration)

Modules:

1. **Message Processing**:

    - Handles user input and sanitizes data before sending it to the AI model.
2. **Response Generation**:
    - Dynamically generates responses using the DialoGPT model.
3. **API Handling**:
    - Provides a REST API endpoint for easy integration with frontend components.

Example Integration:

API Request Flow: 1. User sends a message via the frontend. 2. The message is sent to the /ai-chat/ endpoint. 3. The AI assistant processes the message and generates a response. 4. The response is returned to the user.

Testing and Debugging

Testing Approaches:

1. **Unit Testing**:
    - Test the get_response method of the AIAssistant class.
2. **Integration Testing**:
    - Verify the /ai-chat/ endpoint handles requests and returns expected responses.
3. **Manual Testing**:
    - Simulate user interactions via tools like Postman or the frontend interface.

Debugging Techniques:

- Use print statements during development to log input and output.
- Replace print with proper logging for production:

```
import logging
logger = logging.getLogger(__name__)
logger.info("AI response generated")
```

Additional Notes

Model Customization:

- The AI assistant can be fine-tuned with datasets like **EmpatheticDialogues** to improve response relevance.

Deployment:

- The chatbot can be deployed on cloud platforms like AWS or Azure for scalability.

Future Enhancements:

1. **Sentiment Analysis**:
    - Incorporate sentiment analysis to adjust the tone of AI responses.
2. **Multi-Language Support**:
    - Expand capabilities to handle conversations in multiple languages.
3. **Conversation Context**:
    - Enhance the model to maintain context across multiple interactions.

Frontend Development (neuroAI-client):

GitHub repository: https://github.com/TBS96/neuroAI-client/

Tech Stacks:

Technology	Version	Prerequisites
ReactJS	v18.3.1	Node JS, Git
Vite	v6.0.5	
TailwindCSS	v3.4.17	
ESLint	v9.17.0	
DaisyUI	v4.12.23	
React Hook Form	v7.54.2	
React Router Dom	v6.28.1	
Socket.io-client	v4.8.1	
Express	v4.21.1	**Server side for ChatBot**
CORS	v2.8.5	
{ Server } from socket.io	v4.8.1	

/index.html:

```
<!doctype html>
<html lang="en" class="scroll-smooth" data-theme="light">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="public/neuroAI-icon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>neuroAI</title>
</head>
<body>
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

src/index.css:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

/src/App.jsx:

```
import React, { useState } from 'react'
import { Atom } from 'react-loading-indicators'
import { Container, Footer, Header } from './components';
import { Outlet } from 'react-router-dom';

const App = () => {
const [loading, setLoading] = useState(false);
return !loading ? (
<div>
<div>
<Header />
<Container>
<main>
<Outlet />
</main>
</Container>
<Footer />
</div>
</div>
) : (
<div className='grid place-content-center w-full min-h-screen'>
<Atom color='#5021ec' size='large' />
</div>
)
}

export default App
```

/src/main.jsx:

```
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { createBrowserRouter, RouterProvider } from 'react-router-dom'
import { Home, Login, Signup, Contact, About, ChatBot } from './pages/index.js'

const router = createBrowserRouter([
{
path: '/',
element: <App />,
children: [
{
```

```
path: '/',
element: <Home />
},
{
path: '/login',
element: <Login />
},
{
path: '/signup',
element: <Signup />
},
{
path: '/contact',
element: <Contact />
},
{
path: '/about',
element: <About />
},
{
path: '/chatbot',
element: <ChatBot />
},
]
}
]);

createRoot(document.getElementById('root')).render(
<RouterProvider router={router} />
)
```

COMPONENTS:

/src/components/container/Container.jsx:

```
import React from 'react'

const Container = ({children}) => {
return <div className='w-full max-w-7xl mx-auto px-4'>{children}</div>;
}

export default Container
```

/src/components/Header/Header.jsx:

```
import React, { useState } from 'react'
import { Link, useLocation, useNavigate } from 'react-router-dom'
```

```
import Logo from '../Logo';

function Header() {
const navigate = useNavigate();
const location = useLocation();
const [menubar, setMenubar] = useState(false);
const navItems = [
{
name: 'Home',
slug: '/',
active: true
},
{
name: 'Contact',
slug: '/contact',
active: true
},
{
name: 'About',
slug: '/about',
active: true
},
{
name: 'Login',
slug: '/login',
active: true
},
{
name: 'Signup',
slug: '/signup',
active: true
},
];

return (
<header className='py-1 shadow-2xl shadow-slate-600 bg-blue-900 border-b border-gray-100 backdrop-blur-lg bg-opacity-30 text-white dark:text-gray-200 sticky top-0 z-50'>
<nav className='flex items-center justify-between max-w-7xl mx-auto px-4'>
<div className='mr-4'>
<Link>
<Logo width='100%' />
</Link>
</div>
<button onClick={() => setMenubar(!menubar)} className='btn btn-circle skeleton md:hidden'>
{menubar ? '✘' : '﹀'}
</button>
<ul className={`absolute md:relative top-[72px] left-0 md:top-0 md:left-auto shadow-2xl shadow-slate-600 md:shadow-none bg-blue-900/80 glass md:bg-none backdrop-blur-lg
```

```
md:backdrop-blur-none border-b md:border-none border-gray-100 md:bg-transparent w-full
md:w-auto flex flex-col md:flex-row items-center md:items-center space-y-4 md:space-y-0
md:space-x-6 px-4 py-4 md:px-0 md:py-0 z-50 transition-all duration-300 ${menubar ?
'block' : 'hidden md:flex'}`}>
{navItems.map((item) =>
item.active ? (
<li key={item.name} className={`w-full md:w-auto ${location.pathname === item.slug ?
'bg-black rounded-full' : ''}`}>
<button onClick={() => navigate(item.slug)} className='btn btn-ghost w-full md:w-auto
text-left md:text-center hover:bg-primary px-6 py-2 duration-200 rounded-
full'>{item.name}</button>
</li>
) : null
)}
</ul>
</nav>
</header>
)
}

export default Header
```

/src/components/Footer/Footer.jsx:

```
import React from 'react'
import { Link } from 'react-router-dom'
import { Logo } from '../index'

function Footer() {
const footerLinks = [
{
title: "Social",
links: [
{
name: "GitHub",
path: "https://github.com/tbs96",
icon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"
height="24"
fill="currentColor"
viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
```

d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0 1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.318.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749 1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101 2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679 1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421 10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
},
{
name: 'LinkedIn',
path: 'https://www.linkedin.com/in/prantikghosh96/',
icon: (<svg className='text-blue-500' aria-hidden='true'
xmlns='http://www.w3.org/2000/svg' width='24' height='24' fill='currentColor' viewBox='0 0 24 24'>
<path fill-rule='evenodd' d='M12.51 8.796v1.697a3.738 3.738 0 0 1 3.288-1.684c3.455 0 4.202 2.16 4.202 4.97V19.5h-3.2v-5.072c0-1.21-.244-2.766-2.128-2.766-1.827 0-2.139 1.317-2.139 2.676V19.5h-3.19V8.796h3.168ZM7.2 6.106a1.61 1.61 0 0 1-.988 1.483 1.595 1.595 0 0 1-1.743-.348A1.607 1.607 0 0 1 5.6 4.5a1.601 1.601 0 0 1 1.6 1.606Z' clip-rule='evenodd' />
<path d='M7.2 8.809H4V19.5h3.2V8.809Z' />
</svg>)
},
{
name: 'Email',
path: 'mailto:9tbs6@proton.me',
icon: (<svg className='text-orange-500' aria-hidden='true'
xmlns='http://www.w3.org/2000/svg' width='24' height='24' fill='currentColor' viewBox='0 0 24 24'>
<path d='M17 6h-2V5h1a1 1 0 1 0 0-2h-2a1 1 0 0 0-1 1v2h-.541A5.965 5.965 0 0 1 14 10v4a1 1 0 1 1-2 0v-4c0-2.206-1.794-4-4-4-.075 0-.148.012-.22.028C7.686 6.022 7.596 6 7.5 6A4.505 4.505 0 0 0 3 10.5V16a1 1 0 0 0 1 1h7v3a1 1 0 0 0 1 1h2a1 1 0 0 0 1-1v-3h5a1 1 0 0 0 1-1v-6c0-2.206-1.794-4-4-4Zm-9 8.5H7a1 1 0 1 1 0-2h1a1 1 0 1 1 0 2Z' />
</svg>)
},
{
name: 'X',
path: 'https://x.com/9theblacksheep6',
icon: (<svg className='text-black' aria-hidden='true' xmlns='http://www.w3.org/2000/svg'
width='24' height='24' fill='currentColor' viewBox='0 0 24 24'>
<path d='M13.795 10.533 20.68 2h-3.073l-5.255 6.517L7.69 2H1l7.806 10.91L1.47 22h3.074l5.705-7.07L15.31 22H22l-8.205-11.467Zm-2.38 2.95L9.97 11.464 4.36 3.627h2.31l4.528 6.317 1.443 2.02 6.018 8.409h-2.31l-4.934-6.89Z' />

```
</svg>)
},
{
name: 'Facebook',
path: 'https://www.facebook.com/theblacksheep96/',
icon: (<svg className='text-blue-600' aria-hidden='true'
xmlns='http://www.w3.org/2000/svg' width='24' height='24' fill='currentColor' viewBox='0 0
24 24'>
<path fill-rule='evenodd' d='M13.135 6H15V3h-1.865a4.147 4.147 0 0 0-4.142
4.142V9H7v3h2v9.938h3V12h2.021l.592-3H12V6.591A.6.6 0 0 1 12.592 6h.543Z' clip-
rule='evenodd' />
</svg>)
},
],
},
];

const additionalLinks = [
{
title: "Quick Links",
links: ["Features", "How It Works", "Testimonials"],
},
{
title: "Company",
links: ["About Us", "Remedy", "Contact"],
},
{
title: "Legal",
links: ["Privacy Policy", "Terms of Service", "Cookie Policy"],
},
];

const teamMembers = [
{
name: "Mahvish Ruhi",
url: "https://github.com/Mahvish16",
className: "btn btn-ghost hover:underline underline-offset-4 decoration-error hover:text-
pink-400",
},
{
name: "Sayan Bhattacharya",
url: "https://github.com/Sayan-ezioo",
className: "btn btn-ghost hover:underline underline-offset-4 decoration-amber-500
hover:text-error",
},
{
name: "Prantik Ghosh",
url: "https://github.com/tbs96",
```

```
className: "btn btn-ghost hover:underline underline-offset-4 decoration-green-500
hover:text-yellow-400",
},
{
name: "Sanchita Kar",
url: "https://github.com/",
className: "btn btn-ghost hover:underline underline-offset-4 decoration-pink-500
hover:text-violet-400",
},
{
name: "Arghyakamal Ghosh",
url: "https://github.com/",
className: "btn btn-ghost hover:underline underline-offset-4 decoration-violet-500
hover:text-lime-500",
},
];

return (
<footer className="bg-gray-800 text-white py-10 border-t-2 border-t-gray-700">
<div className="container mx-auto px-4">
{/* Upper Footer */}
<div className="flex flex-wrap justify-between items-center">
{/* Logo */}
<div className="w-full md:w-1/3 text-center border-b md:border-none pb-8 md:pb-0">
<Link to="/">
<Logo width="100px" />
</Link>
</div>

{/* Footer Sections */}
{additionalLinks.map((section, index) => (
<div key={index} className="w-full md:w-auto text-center">
<h3 className="font-semibold text-lg text-gray-400 uppercase my-4 md:my-
0">{section.title}</h3>
<ul className="mt-2">
{section.links.map((link, idx) => (
<li key={idx} className="mt-1 hover:underline underline-offset-4 hover:translate-x-2
transition-all duration-300">
<Link to='/'>
{link}
</Link>
</li>
))}
</ul>
</div>
))}
</div>

<div className="mt-8 text-center border-t border-t-gray-700 pt-4 text-sm"></div>
```

```
{/* Social Links */}
<div className="flex flex-wrap justify-center mt-8">
{footerLinks.map((section) => (
<div key={section.title} className="text-center">
<h3 className="text-gray-400 uppercase font-semibold mb-4">
{section.title}
</h3>
<ul className="md:flex justify-center">
{section.links.map((link) => (
<li key={link.name} className="p-4">
<Link
to={link.path}
className='text-gray-400 btn hover:glass hover:scale-110 rounded-2xl'
target='_blank'
>
{link.icon}
</Link>
</li>
))}
</ul>
</div>
))}
</div>

{/* Lower Footer */}
<div className="mt-8 text-center border-t border-t-gray-700 pt-4 text-sm">
<p>
&copy; 2025{" "}
<Link to="/" className="text-blue-400 hover:text-blue-500">
neuroAI
</Link>{" "}
| All Rights Reserved by{" "}
{teamMembers.map((member, index) => (
<div className="flex md:inline-flex justify-center" key={index}>
<Link
to={member.url}
className={member.className}
target="_blank"
rel="noopener noreferrer"
>
{member.name}
</Link>
{index < teamMembers.length - 1 && " "}
</div>
))}
</p>
</div>
</div>
</footer>
```

```
)
}

export default Footer
```

/src/components/Button.jsx:

```
import React from 'react'

export default function Button({
children,
type = 'button',
bgColor = 'bg-green-500',
textColor = 'text-white',
className = '',
...props
}) {
return (
<button className={`skeleton btn px-4 py-2 rounded-lg transition-all duration-200 $
{bgColor} ${textColor} ${className}`} {...props}>
{children}
</button>
)
}
```

/src/components/Input.jsx:

```
import React, { useId } from 'react'

const Input = React.forwardRef(function Input({
label,
type = 'text',
className = '',
...props
}, ref) {

const id = useId();

return (
<div className='w-full'>
{label &&
<label htmlFor={id} className='inline-block mb-1 pl-1'>
{label}
</label>
}
<input
type={type}
```

```
className={`py-2 rounded-lg text-black outline-none focus:bg-gray-50 duration-200 border
border-gray-200 w-full input glass bg-base-300 ${className}`}
ref={ref}
{...props}
id={id}
/>
</div>
)
})

export default Input
```

/src/components/Logo.jsx:

```
import React from 'react'
import { Link } from 'react-router-dom'

function Logo({width = '100px'}) {
return (
<Link to='/' className=''>
<img
src='/public/favicon/apple-touch-icon.png'
alt='NeuroAI Logo'
className='btn rounded-2xl h-16'
style={{width}}
/>
</Link>
)
}

export default Logo
```

/src/components/Login.jsx:

```
import React, { useState } from 'react'
import { useForm } from 'react-hook-form';
import { Link, useNavigate } from 'react-router-dom'
import { Button, Input, Logo } from './index'

function Login () {

const navigate = useNavigate();
const { register, handleSubmit } = useForm();
const [error, setError] = useState('');
const [data, setData] = useState('');
const [showPass, setShowPass] = useState(false);
const login = () => {};
```

```
return (
<div className='flex items-center justify-center w-full my-8 px-4 sm:px-0'>
<div className={`mx-auto w-full max-w-lg bg-gray-100 rounded-xl p-10 border border-
black/10`}>
<div className='mb-2 flex justify-center'>
<span className='inline-block w-full max-w-[100px]'>
<Logo width='100%' />
</span>
</div>
<h2 className='text-center text-2xl font-bold leading-tight'>Sign in to your account</h2>
<p className='mt-2 text-center text-base text-black/60'>
Don&apos;t have any account? 
<Link to='/signup' className='font-medium text-primary transition-all duration-200
hover:underline'>
Sign Up
</Link>
</p>
{error &&
<p className='text-red-600 mt-8 text-center bg-error-content animate-pulse'>{error}</p>
}
<form onSubmit={handleSubmit(login)} className='mt-8 form-control'>
<div className='space-y-5'>
<Input
label='Email: '
placeholder='example@domain.com'
type='email'
{...register('email', {
required: true,
validate: {
matchPattern: (value) => /^([\w\.\-_]+)?\w+@[\w-_]+(\.\w+){1,}$/.test(value) || 'Email
address must be a valid address',
}
})}
/>
{error &&
<p className='text-red-600 mt-8 text-center animate-pulse bg-red-100'>{data.email} doesn't
exist in our database. Please Sign Up!</p>
}
<div className='flex items-end sm:flex-col gap-2'>
<Input
label='Password: '
placeholder='✪✪✪✪✪✪✪✪✪✪✪✪'
type={showPass ? 'text' : 'password'}
{...register('password', {
required: true,
})}
/>
<input type='checkbox' className='hidden' id='show' onChange={() => setShowPass(!
showPass)} />
```

```
<label htmlFor="show" className='btn btn-square btn-outline'>{showPass ? 'hide' :
'show'}</label>
</div>
<Button type='submit' className='w-full hover:bg-green-600'>Sign in</Button>
</div>
</form>
</div>
</div>
)
}

export default Login
```

/src/components/Signup.jsx:

```
import React, { useState } from 'react'
import { useForm } from 'react-hook-form';
import { Link, useNavigate } from 'react-router-dom'
import { Button, Input, Logo } from './index'

function Signup () {

const navigate = useNavigate();
const { register, handleSubmit } = useForm();
const [error, setError] = useState('');
const [showPass, setShowPass] = useState(false);
const create = () => {};

return (
<div className='flex items-center justify-center w-full my-8 px-4 sm:px-0'>
<div className={`mx-auto w-full max-w-lg bg-gray-100 rounded-xl p-10 border border-
black/10`}>
<div className='mb-2 flex justify-center'>
<span className='inline-block w-full max-w-[100px]'>
<Logo width='100%' />
</span>
</div>
<h2 className='text-center text-2xl font-bold leading-tight'>Sign up to create an
account</h2>
<p className='mt-2 text-center text-base text-black/60'>
Already have an account? 
<Link to='/login' className='font-medium text-primary transition-all duration-200
hover:underline'>
Sign In
</Link>
</p>
{error && <p className='text-red-600 mt-8 text-center'>{error}</p>}
```

```
<form onSubmit={handleSubmit(create)} className='mt-8 form-control'>
<div className='space-y-5'>
<Input
label='Full Name: '
type='text'
placeholder='Full Name'
{...register('name', {
required: true,
})}
/>
<Input
label='Email: '
placeholder='example@domain.com'
type='email'
{...register('email', {
required: true,
validate: {
matchPattern: (value) => /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/.test(value) ||
'Email address must be a valid address',
}
})}
/>
<Input
label='Phone Number: '
placeholder='+91-XXXXXXXXXX'
type='tel'
{...register('phone_number', {
required: true,
})}
/>
<Input
label='Date of Birth: '
type='date'
{...register('dob', {
required: true,
})}
/>
<Input
label='Age: '
placeholder='Age'
type='number'
{...register('age', {
required: true,
})}
/>
<Input
label='Address: '
placeholder='Street Address'
type='text'
```

```
autocomplete='street-address'
{...register('address', {
required: true,
})}
/>
<Input
label='Occupation: '
placeholder='Occupation'
type='text'
{...register('occupation', {
required: true,
})}
/>
<Input
label='Password: '
placeholder='✪✪✪✪✪✪✪✪✪✪'
type={showPass ? 'text' : 'password'}
{...register('password', {
required: true,
})}
/>
<div className='flex items-end sm:flex-col gap-2'>
<Input
label='Confirm Password: '
placeholder='✪✪✪✪✪✪✪✪✪✪'
type={showPass ? 'text' : 'password'}
{...register('password', {
required: true,
})}
/>
<input type='checkbox' className='hidden' id='show' onChange={() => setShowPass(!
showPass)} />
<label htmlFor="'show" className='btn btn-square btn-outline'>{showPass ? 'hide' :
'show'}</label>
</div>
<Button type='submit' className='w-full hover:bg-green-600'>
Create Account
</Button>
</div>
</form>
</div>
</div>
)
}

export default Signup
```

/src/components/ChatBot.jsx: (Client):

```
import React, { useEffect, useState } from 'react'
import io from 'socket.io-client'

let socket;
const CONNECTION_PORT = 'localhost:3001/';

const ChatBot = () => {

const [loggedIn, setLoggedIn] = useState(false);
const [userName, setUserName] = useState('');
const [room, setRoom] = useState('');

const [message, setMessage] = useState('');
const [messageList, setMessageList] = useState([]);

useEffect(() => {
socket = io(CONNECTION_PORT)
}, [CONNECTION_PORT]);

useEffect(() => {
socket.on('receive_message', (data) => {
setMessageList((prevList) => [...prevList, data]);
})
}, []);

const connectToRoom = () => {
setLoggedIn(true);
socket.emit('join_room', room);
};

const sendMessage = async () => {

if (message.trim() !== '') {
let messageContent = {
room: room,
content: {
author: userName,
message: message
},
};

await socket.emit('send_message', messageContent);
setMessageList([...messageList, messageContent.content]);
setMessage('');
}
};

return (
```

```
<div className='grid place-items-center h-screen bg-[#2a2730]'>
{!loggedIn ? (
<div className='w-[600px] h-[350px] border-4 border-blue-600 rounded-lg flex justify-center items-center flex-col'>

<div className='text-gray-200'>
<input type="text" placeholder='Name...' onChange={(e) =>
{ setUserName(e.target.value) }} className='m-10 w-52 h-10 border-2 border-[#0091ff] bg-transparent focus:outline-none pl-2 text-lg rounded-lg' />
<input type="text" placeholder='Room...' onChange={(e) => { setRoom(e.target.value) }}
className='m-10 w-52 h-10 border-2 border-[#0091ff] bg-transparent focus:outline-none pl-2 text-lg rounded-lg' />
</div>

<button onClick={connectToRoom} className='w-52 h-12 bg-[#0091ff] text-white text-lg mt-11 hover:bg-[#037edc] rounded-lg'>Enter Chat</button>

</div>
) : (
// chatContainer
<div className='w-[600px] h-[350px] border-4 border-blue-600 rounded-lg flex flex-col text-gray-300'>

{/* messages */}
<div className='flex-[80%] w-full overflow-y-auto p-4 space-y-2'>
{messageList.map((val, key) => {
const isCurrentUSer = val.author === userName;
return (
// messageContainer
<div key={key} className={`flex ${isCurrentUSer ? 'justify-end' : 'justify-start'} my-2`}
id={isCurrentUSer ? 'You' : val.author}>
{' '}
<div className={`max-w-xs p-3 rounded-lg ${isCurrentUSer ? 'bg-blue-500 text-white self-end' : 'bg-gray-300 text-black self-start'}`}>
<span className='block font-semibold'>{isCurrentUSer ? 'You' : val.author}</span>
<span>{val.message}</span>
</div>
</div>
)
})}
</div>

{/* messageInputs */}
<div className='flex-[20%] flex flex-row'>
<input type="text" onChange={(e) => { setMessage(e.target.value) }} value={message}
className='flex-[80%] h-[calc(100%-5px)] border-t-4 border-[#0091ff] bg-transparent focus:outline-none text-lg rounded-l-lg pl-2' placeholder='Type a Message...' />
<button onClick={sendMessage} className='flex-[20%] h-full bg-[#16c525] border-t-4 border-[#0091ff] text-white text-lg hover:bg-[#16c525cb] rounded-r-lg'>Send</button>
```

```
</div>

</div>
)}
</div>
)
}

export default ChatBot
```

/neuroAI-server/index.js: (ChatBot Server):

```
import express from 'express'
import cors from 'cors'
import { Server } from 'socket.io'

const app = express();
app.use(cors());
app.use(express.json());
const PORT = 3001;

const server = app.listen(PORT, () => {
   console.log(`Server running on http://localhost:${PORT}`)
});

const io = new Server(server, {
   cors: {
      origin: '*',
      methods: ['GET', 'POST']
   }
});

io.on('connect', (socket) => {
   console.log(`Socket ID: ${socket.id}`)

   socket.on('join_room', (data) => {
      socket.join(data)
      console.log(`User Joined Room: ${data}`)
   })

   socket.on('send_message', (data) => {
      console.log(data)
      socket.to(data.room).emit('receive_message', data.content)
   })

   socket.on('disconnect', () => {
```

```
        console.log('USER DISCONNECTED')
    })
});
```

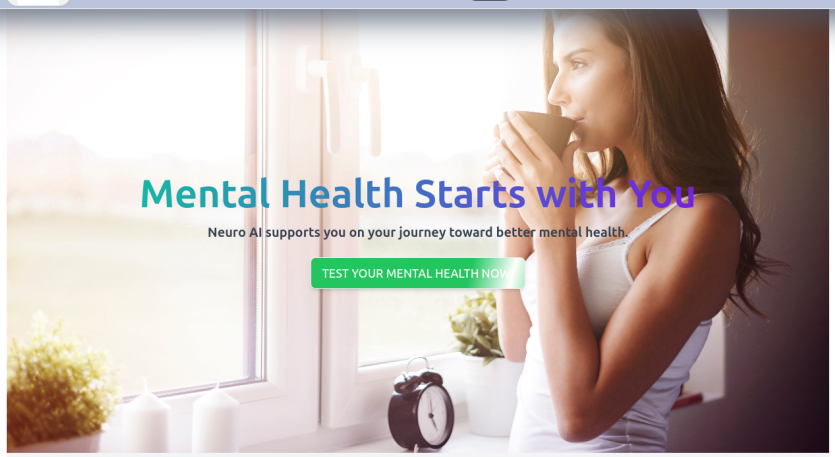
/src/components/index.js:

```
import Input from "./Input";
import Header from "./Header/Header";
import Container from "./container/Container";
import Button from "./Button";
import Footer from "./Footer/Footer";
import Login from './Login';
import Signup from "./Signup";
import Logo from "./Logo";
import ChatBot from "./ChatBot";

export {
Input,
Header,
Container,
Button,
Footer,
Login,
Signup,
Logo,
ChatBot,
}
```

PAGES:

/src/pages/Home.jsx:

Home Contact About Login Signup



Mental Health Starts with You

Neuro AI supports you on your journey toward better mental health.

TEST YOUR MENTAL HEALTH NOW!

A better way to improve your mental health

Evidence-based

Our approach is grounded in scientific research and proven methodologies.

Personalized Care

Tailored support that adapts to your unique needs and progress.

Instant Access

24/7 support at your fingertips, whenever and wherever you need it.

How It Works

1

Sign Up

Create your account and complete a brief assessment.

2

Test Now!

Test to get a better mental health

3

Get Result

Get to know about your concerns towards better mental health.

What Our Users Say

'Neuro AI has been a game-changer for my mental health. The personalized approach and constant support have made a real difference in my life.'

Sarah J.
Neuro AI User

'I was skeptical at first, but Neuro AI exceeded my expectations. It's like having a therapist in your pocket, available whenever you need it.'

Michael T.
Neuro AI User

QUICK LINKS
Features
How It Works
Testimonials

COMPANY
About Us
Remedy
Contact

LEGAL
Privacy Policy
Terms of Service
Cookie Policy

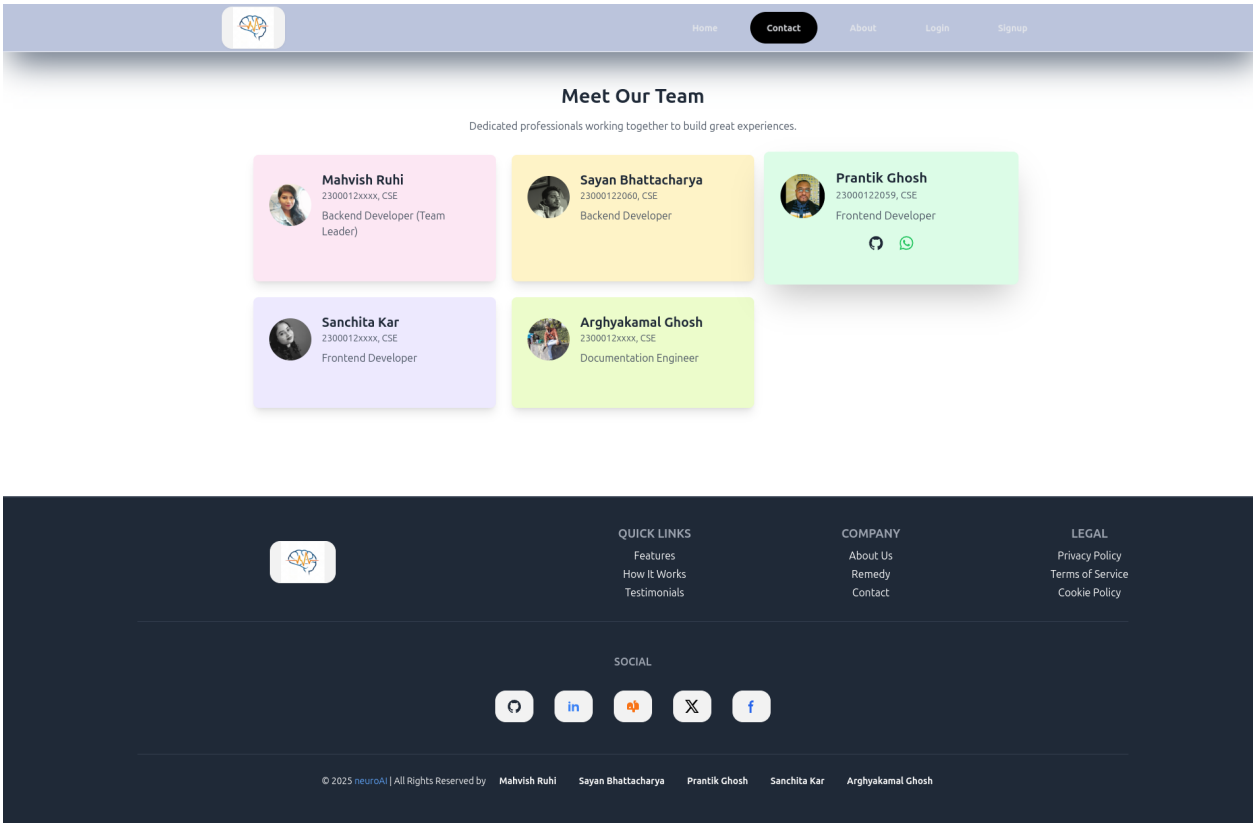
SOCIAL

```
import React from 'react'
import { Link } from 'react-router-dom'
import { Button } from '../components/index'

const Home = () => {
return (
<main>
<section
className='relative h-[100vh] bg-cover bg-no-repeat bg-center flex items-center justify-center'
style={{ backgroundImage: "url('https://i.ibb.co/3kq40pK/home-hero.jpg')" }}>
<div className='text-center px-6 md:px-12'>
<h1 className='bg-clip-text text-transparent bg-gradient-to-r from-[#14b8a6] to-[#7520e4] text-4xl md:text-6xl font-bold'>
Mental Health Starts with You
</h1>
<p className='mt-4 text-gray-700 text-lg font-bold md:text-xl'>
Neuro AI supports you on your journey toward better mental health.
</p>
<div className='mt-6'>
<Link to='/chatbot'>
<Button className='bg-[#14b8a6] text-white font-medium text-lg shadow-md hover:bg-[#128a7d] transition duration-300'>
TEST YOUR MENTAL HEALTH NOW!
</Button>
</Link>
</div>
</div>
</section>

<section className='py-16 bg-gray-100' id='features'>
<div className='container mx-auto px-6 md:px-12 text-center'>
<h2 className='text-3xl md:text-4xl font-semibold'>
A better way to improve your mental health
</h2>
<div className='mt-12 grid gap-8 md:grid-cols-3'>
{[
{
img: 'https://i.ibb.co/FBXY7FJ/istockphoto2.jpg%22',
title: 'Evidence-based',
desc: 'Our approach is grounded in scientific research and proven methodologies.',
},
{
img: 'https://i.ibb.co/DYk6JHT/istockphoto-3.jpg',
title: 'Personalized Care',
desc: 'Tailored support that adapts to your unique needs and progress.',
},
{
img: 'https://i.ibb.co/qB9ncpV/istockphoto.jpg',
```

```
title: 'Instant Access',
desc: '24/7 support at your fingertips, whenever and wherever you need it.',
},
].map((feature, index) => (
<div key={index} className='bg-white p-6 shadow-md hover:shadow-2xl transition-all
duration-300 hover:scale-105 hover:cursor-pointer rounded-md text-left'>
<img
src={feature.img}
alt={feature.title}
className='w-full h-48 object-cover rounded-md mb-4'
/>
<h3 className='text-2xl font-medium'>{feature.title}</h3>
<p className='mt-2 text-gray-600'>{feature.desc}</p>
</div>
))}
</div>
</div>
</section>

<section className='py-16 bg-white'>
<div className='container mx-auto px-6 md:px-12 text-center'>
<h2 className='text-3xl md:text-4xl font-semibold'>How It Works</h2>
<div className='mt-12 grid gap-8 md:grid-cols-3'>
{[
{
number: '1',
title: 'Sign Up',
desc: 'Create your account and complete a brief assessment.',
},
{
number: '2',
title: 'Test Now!',
desc: 'Test to get a better mental health',
},
{
number: '3',
title: 'Get Result',
desc: 'Get to know about your concerns towards better mental health.',
},
].map((step, index) => (
<div key={index} className='flex flex-col items-center bg-gray-100 p-6 rounded-md
shadow-md hover:shadow-2xl transition-all duration-300 hover:translate-x-4 hover:cursor-
pointer'>
<div className='text-4xl font-bold bg-[#14b8a6] text-white w-16 h-16 flex items-center
justify-center rounded-full mb-4'>
{step.number}
</div>
<h3 className='text-2xl font-medium'>{step.title}</h3>
<p className='mt-2 text-gray-600'>{step.desc}</p>
```

```
</div>
))}
</div>
</div>
</section>

{/* Testimonials Section */}
<section className='py-16 bg-gray-100' id='testimonials'>
<div className='container mx-auto px-6 md:px-12 text-center'>
<h2 className='text-3xl md:text-4xl font-semibold'>
What Our Users Say
</h2>
<div className='mt-12 grid gap-8 md:grid-cols-2'>
{[
{
quote:
'Neuro AI has been a game-changer for my mental health. The personalized approach and constant support have made a real difference in my life.',
author: 'Sarah J.',
role: 'Neuro AI User',
},
{
quote:
"I was skeptical at first, but Neuro AI exceeded my expectations. It's like having a therapist in your pocket, available whenever you need it.",
author: 'Michael T.',
role: 'Neuro AI User',
},
].map((testimonial, index) => (
<div key={index} className='bg-white p-6 shadow-md rounded-md text-left hover:shadow-2xl transition-all duration-300 hover:rotate-2 hover:cursor-pointer'>
<p className='italic text-gray-700'>'{testimonial.quote}'</p>
<div className='mt-4'>
<h4 className='text-lg font-medium'>{testimonial.author}</h4>
<p className='text-sm text-gray-500'>{testimonial.role}</p>
</div>
</div>
))}
</div>
</div>
</section>

</main>
)
}

export default Home
```

/src/pages/Contact.jsx




```
import React from 'react'
import { Link } from 'react-router-dom';

const teamMembers = [
{
name: "Mahvish Ruhi",
roll: '2300012xxxx, CSE',
role: "Backend Developer (Team Leader)",
github: "https://github.com/Mahvish16",
githubIcon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"
height="24"
fill="currentColor"
viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0 1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-
```

1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.31
8.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-
1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749
1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101
2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679
1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421
10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
wap: 'https://wa.me/7070067301',
wapIcon: (
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24
24"><path fill="currentColor" d="M19.05 4.91A9.82 9.82 0 0 0 12.04 2c-5.46 0-9.91 4.45-
9.91 9.91c0 1.75.46 3.45 1.32 4.95L2.05 22l5.25-1.38c1.45.79 3.08 1.21 4.74 1.21c5.46 0
9.91-4.45 9.91-9.91c0-2.65-1.03-5.14-2.9-7.01m-7.01 15.24c-1.48 0-2.93-.4-4.2-
1.15l-.3-.18l-3.12.82l.83-3.04l-.2-.31a8.26 8.26 0 0 1-1.26-4.38c0-4.54 3.7-8.24 8.24-
8.24c2.2 0 4.27.86 5.82 2.42a8.18 8.18 0 0 1 2.41 5.83c.02 4.54-3.68 8.23-8.22 8.23m4.52-
6.16c-.25-.12-1.47-.72-
1.69-.81c-.23-.08-.39-.12-.56.12c-.17.25-.64.81-.78.97c-.14.17-.29.19-.54.06c-.25-.12-
1.05-.39-1.99-1.23c-.74-.66-1.23-1.47-1.38-
1.72c-.14-.25-.02-.38.11-.51c.11-.11.25-.29.37-.43s.17-.25.25-.41c.08-.17.04-.31-.02-.43s-.56
-1.34-.76-1.84c-.2-.48-.41-.42-.56-.43h-.48c-.17 0-.43.06-.66.31c-.22.25-.86.85-.86 2.07s.89
2.4 1.01 2.56c.12.17 1.75 2.67 4.23 3.74c.59.26 1.05.41 1.41.52c.59.19 1.13.16
1.56.1c.48-.07 1.47-.6 1.67-1.18c.21-.58.21-1.07.14-1.18s-.22-.16-.47-.28"/></svg>
),
color: "bg-pink-100",
image: "https://avatars.githubusercontent.com/u/122742962?v=4",
},
{
name: "Sayan Bhattacharya",
roll: '23000122060, CSE',
role: "Backend Developer",
github: "https://github.com/Sayan-ezioo",
githubIcon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"
height="24"
fill="currentColor"
viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0
1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-

1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-
1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.31
8.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-
1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749
1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101
2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679
1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421
10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
wap: 'https://wa.me/8240127576',
wapIcon: (
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24
24"><path fill="currentColor" d="M19.05 4.91A9.82 9.82 0 0 0 12.04 2c-5.46 0-9.91 4.45-
9.91 9.91c0 1.75.46 3.45 1.32 4.95L2.05 22l5.25-1.38c1.45.79 3.08 1.21 4.74 1.21c5.46 0
9.91-4.45 9.91-9.91c0-2.65-1.03-5.14-2.9-7.01m-7.01 15.24c-1.48 0-2.93-.4-4.2-
1.15l-.3-.18l-3.12.82l.83-3.04l-.2-.31a8.26 8.26 0 0 1-1.26-4.38c0-4.54 3.7-8.24 8.24-
8.24c2.2 0 4.27.86 5.82 2.42a8.18 8.18 0 0 1 2.41 5.83c.02 4.54-3.68 8.23-8.22 8.23m4.52-
6.16c-.25-.12-1.47-.72-
1.69-.81c-.23-.08-.39-.12-.56.12c-.17.25-.64.81-.78.97c-.14.17-.29.19-.54.06c-.25-.12-
1.05-.39-1.99-1.23c-.74-.66-1.23-1.47-1.38-
1.72c-.14-.25-.02-.38.11-.51c.11-.11.25-.29.37-.43s.17-.25.25-.41c.08-.17.04-.31-.02-.43s-.56
-1.34-.76-1.84c-.2-.48-.41-.42-.56-.43h-.48c-.17 0-.43.06-.66.31c-.22.25-.86.85-.86 2.07s.89
2.4 1.01 2.56c.12.17 1.75 2.67 4.23 3.74c.59.26 1.05.41 1.41.52c.59.19 1.13.16
1.56.1c.48-.07 1.47-.6 1.67-1.18c.21-.58.21-1.07.14-1.18s-.22-.16-.47-.28"/></svg>
),
color: "bg-amber-100",
image: "https://scontent.fccu18-1.fna.fbcdn.net/v/t39.30808-
6/332876237_2226067160909699_5947686179672796385_n.jpg?_nc_cat=104&ccb=1-
7&_nc_sid=6ee11a&_nc_ohc=v3c-
LiPQHYgQ7kNvgFxcECs&_nc_zt=23&_nc_ht=scontent.fccu18-
1.fna&_nc_gid=AcYIbRIvoGy020gXSSplJnR&oh=00_AYBOG8ZgNcCxyp5iSRx5W8vRq
3WOGyeRFGjPQxHr0-QePQ&oe=6790CEC7",
},
{
name: "Prantik Ghosh",
roll: '23000122059, CSE',
role: "Frontend Developer",
github: "https://github.com/tbs96",
githubIcon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"
height="24"
fill="currentColor"

viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0 1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.318.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749 1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101 2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679 1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421 10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
wap: 'https://wa.me/8910325359',
wapIcon: (
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24"><path fill="currentColor" d="M19.05 4.91A9.82 9.82 0 0 0 12.04 2c-5.46 0-9.91 4.45-9.91 9.91c0 1.75.46 3.45 1.32 4.95L2.05 22l5.25-1.38c1.45.79 3.08 1.21 4.74 1.21c5.46 0 9.91-4.45 9.91-9.91c0-2.65-1.03-5.14-2.9-7.01m-7.01 15.24c-1.48 0-2.93-.4-4.2-1.15l-.3-.18l-3.12.82l.83-3.04l-.2-.31a8.26 8.26 0 0 1-1.26-4.38c0-4.54 3.7-8.24 8.24-8.24c2.2 0 4.27.86 5.82 2.42a8.18 8.18 0 0 1 2.41 5.83c.02 4.54-3.68 8.23-8.22 8.23m4.52-6.16c-.25-.12-1.47-.72-1.69-.81c-.23-.08-.39-.12-.56.12c-.17.25-.64.81-.78.97c-.14.17-.29.19-.54.06c-.25-.12-1.05-.39-1.99-1.23c-.74-.66-1.23-1.47-1.38-1.72c-.14-.25-.02-.38.11-.51c.11-.11.25-.29.37-.43s.17-.25.25-.41c.08-.17.04-.31-.02-.43s-.56-1.34-.76-1.84c-.2-.48-.41-.42-.56-.43h-.48c-.17 0-.43.06-.66.31c-.22.25-.86.85-.86 2.07s.89 2.4 1.01 2.56c.12.17 1.75 2.67 4.23 3.74c.59.26 1.05.41 1.41.52c.59.19 1.13.16 1.56.1c.48-.07 1.47-.6 1.67-1.18c.21-.58.21-1.07.14-1.18s-.22-.16-.47-.28"/></svg>
),
color: "bg-green-100",
image: "https://avatars.githubusercontent.com/u/118633815?v=4",
},
{
name: "Sanchita Kar",
roll: '2300012xxxx, CSE',
role: "Frontend Developer",
github: "https://github.com/",
githubIcon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"
height="24"

fill="currentColor"
viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0 1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.318.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749 1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101 2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679 1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421 10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
wap: 'https://wa.me/7908682921',
wapIcon: (
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24"><path fill="currentColor" d="M19.05 4.91A9.82 9.82 0 0 0 12.04 2c-5.46 0-9.91 4.45-9.91 9.91c0 1.75.46 3.45 1.32 4.95L2.05 22l5.25-1.38c1.45.79 3.08 1.21 4.74 1.21c5.46 0 9.91-4.45 9.91-9.91c0-2.65-1.03-5.14-2.9-7.01m-7.01 15.24c-1.48 0-2.93-.4-4.2-1.15l-.3-.18l-3.12.82l.83-3.04l-.2-.31a8.26 8.26 0 0 1-1.26-4.38c0-4.54 3.7-8.24 8.24-8.24c2.2 0 4.27.86 5.82 2.42a8.18 8.18 0 0 1 2.41 5.83c.02 4.54-3.68 8.23-8.22 8.23m4.52-6.16c-.25-.12-1.47-.72-1.69-.81c-.23-.08-.39-.12-.56.12c-.17.25-.64.81-.78.97c-.14.17-.29.19-.54.06c-.25-.12-1.05-.39-1.99-1.23c-.74-.66-1.23-1.47-1.38-1.72c-.14-.25-.02-.38.11-.51c.11-.11.25-.29.37-.43s.17-.25.25-.41c.08-.17.04-.31-.02-.43s-.56-1.34-.76-1.84c-.2-.48-.41-.42-.56-.43h-.48c-.17 0-.43.06-.66.31c-.22.25-.86.85-.86 2.07s.89 2.4 1.01 2.56c.12.17 1.75 2.67 4.23 3.74c.59.26 1.05.41 1.41.52c.59.19 1.13.16 1.56.1c.48-.07 1.47-.6 1.67-1.18c.21-.58.21-1.07.14-1.18s-.22-.16-.47-.28"/></svg>
),
color: "bg-violet-100",
image: "https://i.ibb.co/VWPdx3V/sanchita-wp.png",
},
{
name: "Arghyakamal Ghosh",
roll: '2300012xxxx, CSE',
role: "Documentation Engineer",
github: "https://github.com/",
githubIcon: (
<svg
className="text-gray-800"
aria-hidden="true"
xmlns="http://www.w3.org/2000/svg"
width="24"

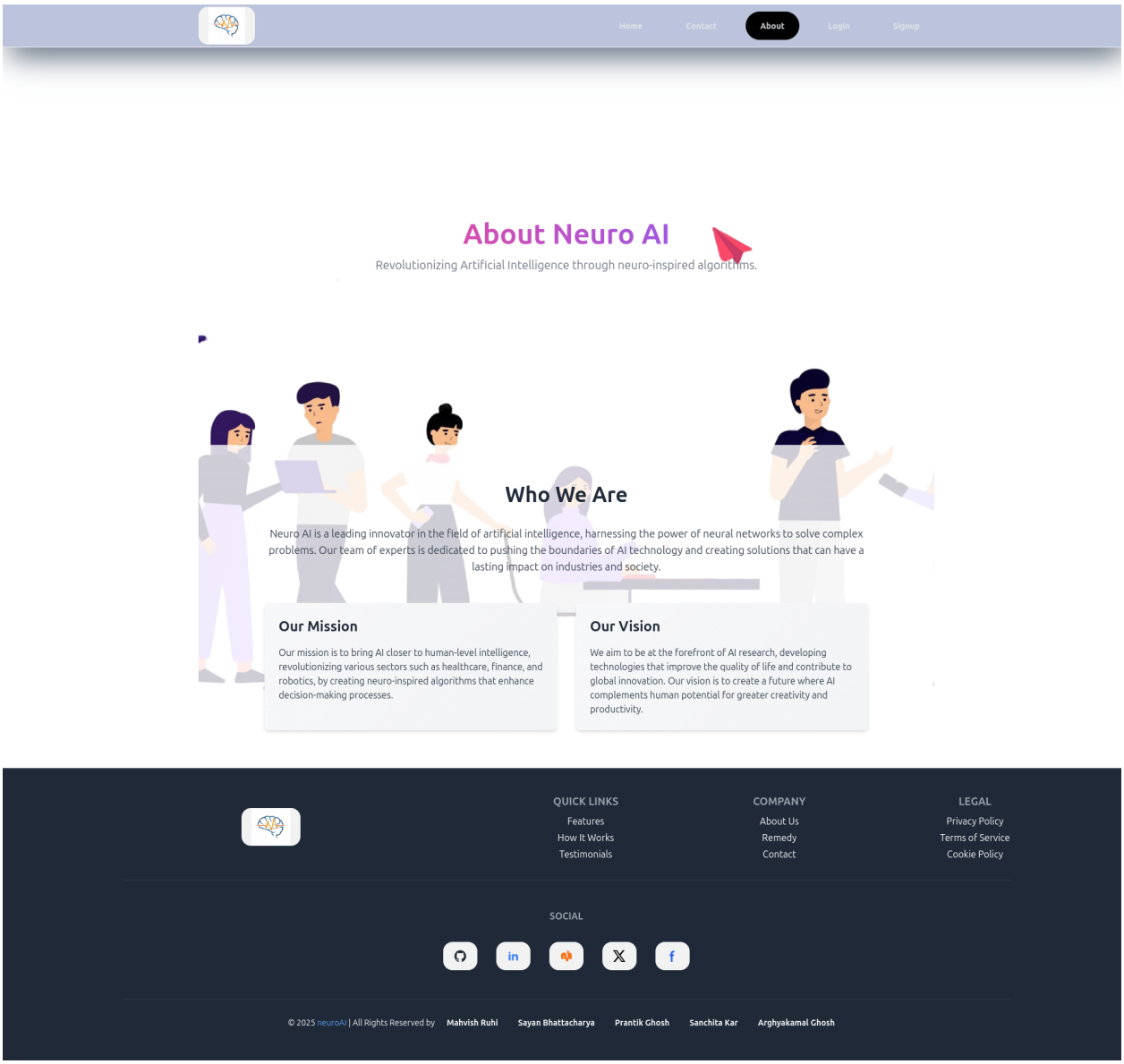
```
height="24"
fill="currentColor"
viewBox="0 0 24 24"
>
<path
fillRule="evenodd"
d="M12.006 2a9.847 9.847 0 0 0-6.484 2.44 10.32 10.32 0 0 0-3.393 6.17 10.48 10.48 0 0 0
1.317 6.955 10.045 10.045 0 0 0 5.4 4.418c.504.095.683-.223.683-.494 0-.245-.01-
1.052-.014-1.908-2.78.62-3.366-1.21-3.366-1.21a2.711 2.711 0 0 0-1.11-
1.5c-.907-.637.07-.621.07-.621.317.044.62.163.885.346.266.183.487.426.647.71.135.253.31
8.476.538.655a2.079 2.079 0 0 0 2.37.196c.045-.52.27-1.006.635-1.37-2.219-.259-4.554-
1.138-4.554-5.07a4.022 4.022 0 0 1 1.031-2.75 3.77 3.77 0 0 1 .096-2.713s.839-.275 2.749
1.05a9.26 9.26 0 0 1 5.004 0c1.906-1.325 2.74-1.05 2.74-1.05.37.858.406 1.828.101
2.713a4.017 4.017 0 0 1 1.029 2.75c0 3.939-2.339 4.805-4.564 5.058a2.471 2.471 0 0 1 .679
1.897c0 1.372-.012 2.477-.012 2.814 0 .272.18.592.687.492a10.05 10.05 0 0 0 5.388-4.421
10.473 10.473 0 0 0 1.313-6.948 10.32 10.32 0 0 0-3.39-6.165A9.847 9.847 0 0 0 12.007 2Z"
clipRule="evenodd"
/>
</svg>
),
wap: 'https://wa.me/8170988792',
wapIcon: (
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24
24"><path fill="currentColor" d="M19.05 4.91A9.82 9.82 0 0 0 12.04 2c-5.46 0-9.91 4.45-
9.91 9.91c0 1.75.46 3.45 1.32 4.95L2.05 22l5.25-1.38c1.45.79 3.08 1.21 4.74 1.21c5.46 0
9.91-4.45 9.91-9.91c0-2.65-1.03-5.14-2.9-7.01m-7.01 15.24c-1.48 0-2.93-.4-4.2-
1.15l-.3-.18l-3.12.82l.83-3.04l-.2-.31a8.26 8.26 0 0 1-1.26-4.38c0-4.54 3.7-8.24 8.24-
8.24c2.2 0 4.27.86 5.82 2.42a8.18 8.18 0 0 1 2.41 5.83c.02 4.54-3.68 8.23-8.22 8.23m4.52-
6.16c-.25-.12-1.47-.72-
1.69-.81c-.23-.08-.39-.12-.56.12c-.17.25-.64.81-.78.97c-.14.17-.29.19-.54.06c-.25-.12-
1.05-.39-1.99-1.23c-.74-.66-1.23-1.47-1.38-
1.72c-.14-.25-.02-.38.11-.51c.11-.11.25-.29.37-.43s.17-.25.25-.41c.08-.17.04-.31-.02-.43s-.56
-1.34-.76-1.84c-.2-.48-.41-.42-.56-.43h-.48c-.17 0-.43.06-.66.31c-.22.25-.86.85-.86 2.07s.89
2.4 1.01 2.56c.12.17 1.75 2.67 4.23 3.74c.59.26 1.05.41 1.41.52c.59.19 1.13.16
1.56.1c.48-.07 1.47-.6 1.67-1.18c.21-.58.21-1.07.14-1.18s-.22-.16-.47-.28"/></svg>
),
color: "bg-lime-100",
image: "https://scontent.fccu18-1.fna.fbcdn.net/v/t39.30808-
6/325335197_715415683558888_4434923817906332409_n.jpg?_nc_cat=105&ccb=1-
7&_nc_sid=6ee11a&_nc_ohc=jdStljGbHuQQ7kNvgFFTe9o&_nc_zt=23&_nc_ht=scontent.f
ccu18-
1.fna&_nc_gid=A0XxxqWD3hN_1LpzqWsnKjQ&oh=00_AYADpwzdNQOh9zFVJJNJSQn
RdD2P106t1e-5x4JrRS65gQ&oe=6790C43D",
},
];

const Contact = () => {
return (
<div className="py-12 min-h-screen">
```

```
{/* TODO */}
{/* <div className='bg-cover bg-no-repeat bg-center min-h-screen relative'
style={{ backgroundImage: "url('public/contact.svg')", filter: "blur(8px)" }}></div> */}
<div className="container mx-auto px-6 sm:px-12">
<h2 className="text-3xl font-bold text-center text-gray-800">
Meet Our Team
</h2>
<p className="mt-4 text-center text-gray-600">
Dedicated professionals working together to build great experiences.
</p>
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6 mt-8">
{teamMembers.map((member, index) => (
<div
key={index}
className={`group p-6 rounded-lg shadow-lg ${member.color} hover:shadow-2xl
hover:shadow-[${member.color}] hover:scale-105 transition`}
>
<div className="flex items-center space-x-4">
<img
src={member.image}
alt={member.name}
className="w-16 h-16 rounded-full object-cover"
/>
<div>
<h3 className="text-xl font-semibold text-gray-800">
{member.name}
</h3>
<p className="text-gray-600 text-sm">{member.roll}</p>
<p className="mt-2 text-gray-600">{member.role}</p>
</div>
</div>
<div className='flex justify-center items-center gap-5'>
<Link
to={member.github}
target="_blank"
rel="noopener noreferrer"
className="mt-4 inline-block text-sm text-blue-600 underline opacity-0 group-
hover:opacity-100 transition-opacity"
>
{member.githubIcon}
</Link>
<Link
to={member.wap}
target="_blank"
rel="noopener noreferrer"
className="mt-4 inline-block text-sm text-green-500 underline opacity-0 group-
hover:opacity-100 transition-opacity"
>
{member.wapIcon}
```

```
</Link>
</div>
</div>
))}
</div>
</div>
</div>
)
}
```

export default Contact

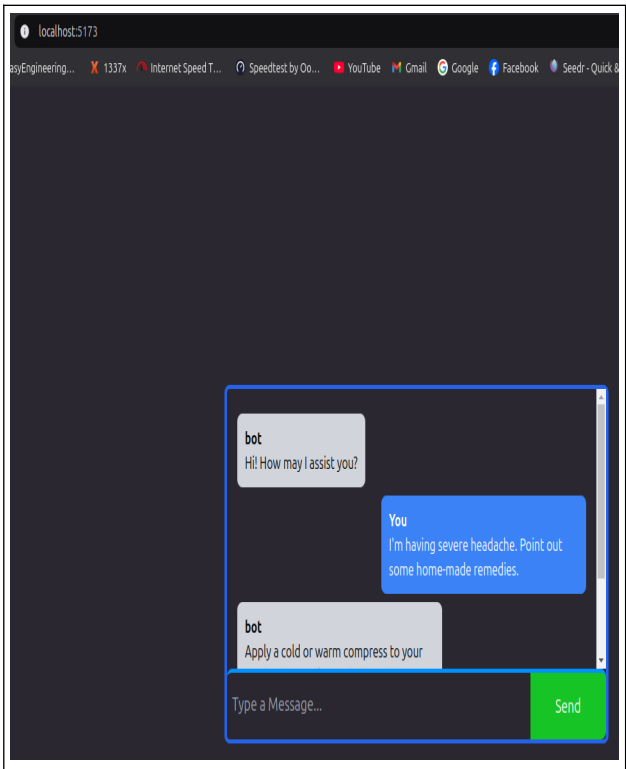
/src/pages/About.jsx:

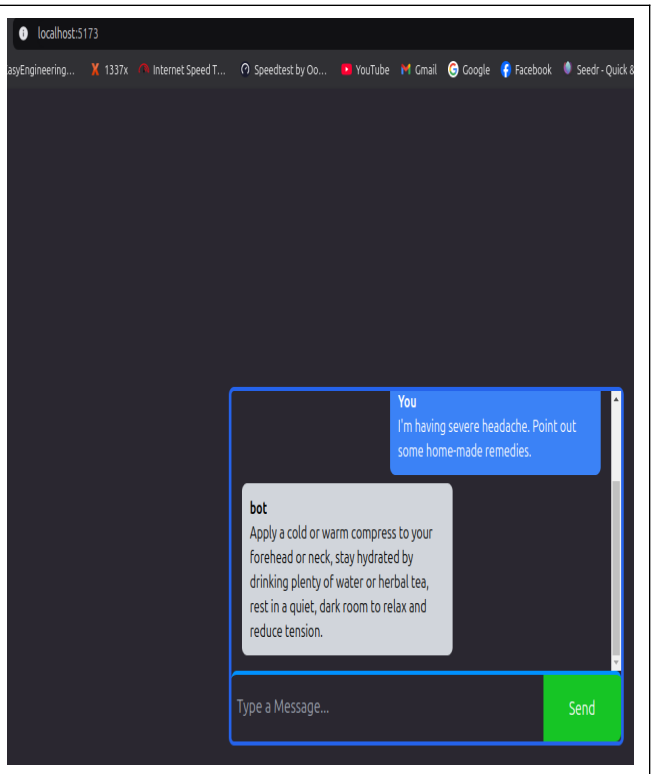


```
import React from 'react'

const About = () => {
return (
<div className='bg-cover bg-center bg-no-repeat' style={{ backgroundImage:
"url('https://i.ibb.co/h1ms14V/team.png')" }}>

<section className='flex items-center justify-center min-h-screen text-gray-500 text-center
py-16'>
<div className='px-4'>
<h2 className='text-4xl sm:text-5xl font-bold mb-4 bg-clip-text text-transparent bg-
gradient-to-r from-pink-500 to-violet-500'>About Neuro AI</h2>
<p className='text-lg sm:text-xl'>
Revolutionizing Artificial Intelligence through neuro-inspired algorithms.
</p>
</div>
</section>

<section className='bg-white bg-opacity-80 py-16'>
<div className='max-w-6xl mx-auto px-6 md:px-12 lg:px-16'>
<h2 className='text-3xl sm:text-4xl font-semibold text-gray-800 text-center mb-8'>Who We
Are</h2>
<p className='text-gray-700 text-base sm:text-lg text-center mb-12'>
Neuro AI is a leading innovator in the field of artificial intelligence, harnessing the power of
neural networks to solve complex problems. Our team of experts is dedicated to pushing the
boundaries of AI technology and creating solutions that can have a lasting impact on
industries and society.
</p>

<div className='grid grid-cols-1 md:grid-cols-2 gap-8'>
<div className='bg-gray-100 glass hover:skeleton hover:backdrop-blur-sm p-6 rounded-lg
shadow-md hover:shadow-xl hover:scale-105 duration-200 cursor-pointer'>
<h3 className='text-2xl font-semibold text-gray-800 mb-4'>Our Mission</h3>
<p className='text-gray-700'>
Our mission is to bring AI closer to human-level intelligence, revolutionizing various sectors
such as healthcare, finance, and robotics, by creating neuro-inspired algorithms that enhance
decision-making processes.
</p>
</div>

<div className='bg-gray-100 glass hover:skeleton hover:backdrop-blur-sm p-6 rounded-lg
shadow-md hover:shadow-xl hover:scale-105 duration-200 cursor-pointer'>
<h3 className='text-2xl font-semibold text-gray-800 mb-4'>Our Vision</h3>
<p className='text-gray-700'>
We aim to be at the forefront of AI research, developing technologies that improve the
quality of life and contribute to global innovation. Our vision is to create a future where AI
complements human potential for greater creativity and productivity.
</p>
</div>
```

```
</div>
</div>
</section>

</div>
)
}

export default About
```

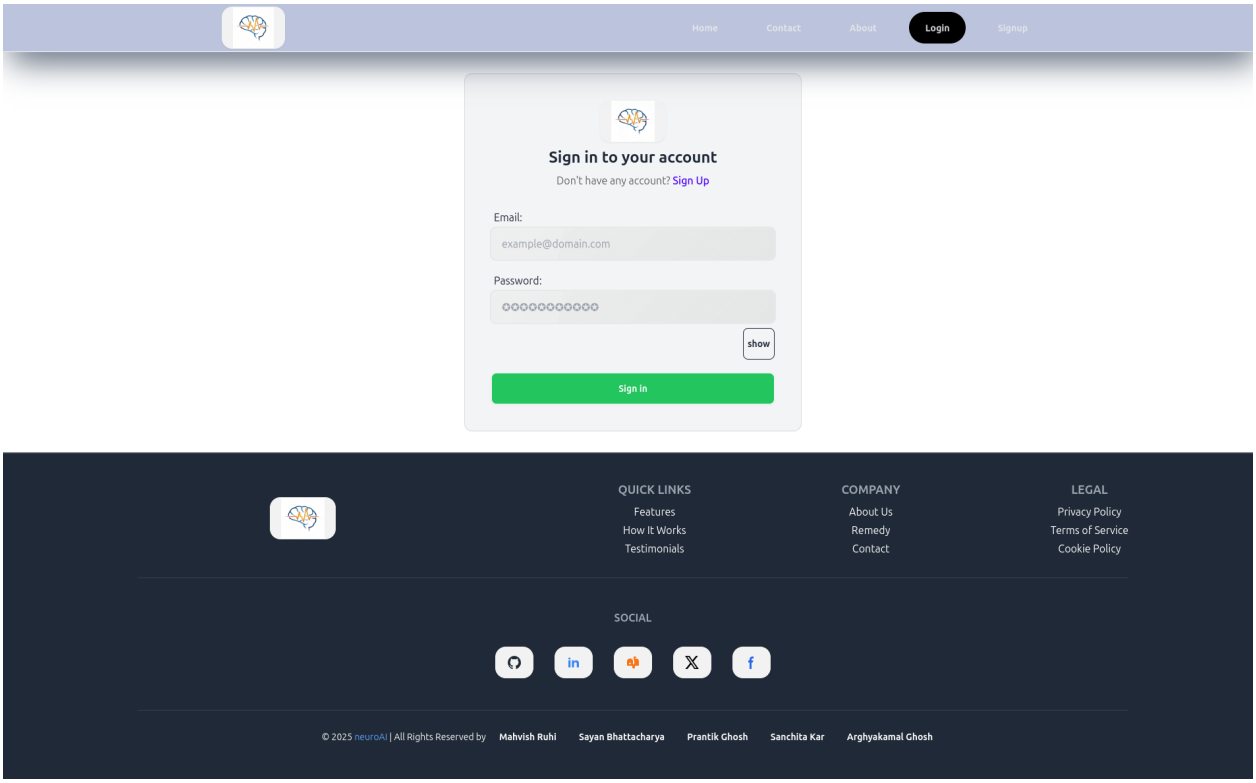
/src/pages/ChatBot.jsx:

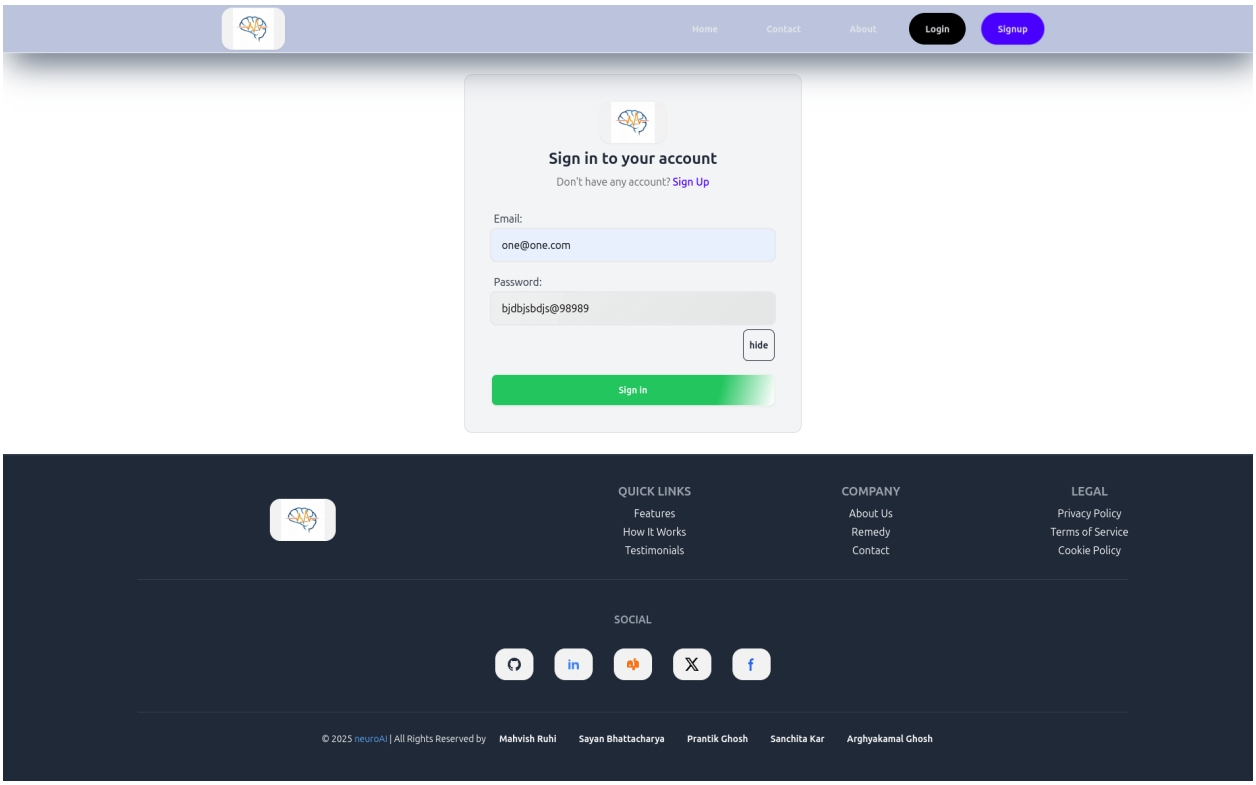





```
import React from 'react'
import { ChatBot as ChatbotComponent } from '../components/index'

function ChatBot() {
return (
<div>
<ChatbotComponent></ChatbotComponent>
</div>
)
}

export default ChatBot
```

/src/pages/Login.jsx:





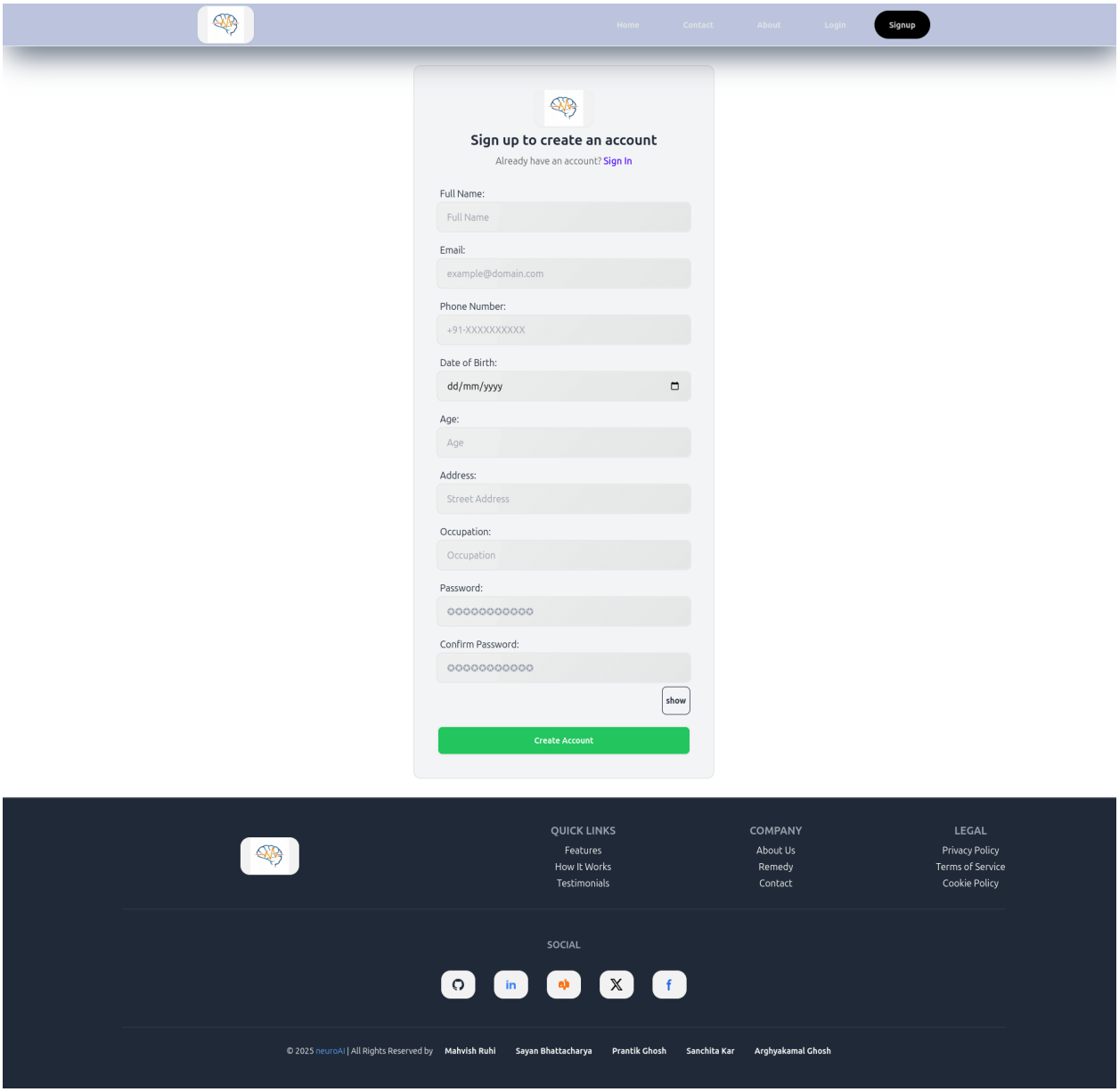

```
import React from 'react'
import { Login as LoginComponent } from '../components'

function Login() {
return (
<div>
```

<LoginComponent />

)
}

export default Login

/src/pages/Signup.jsx:



import React from 'react'
import { Signup as SignupComponent } from '../components'

function Signup() {
return (

```
<div>
<SignupComponent />
</div>
)
}

export default Signup
```

/src/pages/index.js:

```
import Home from './Home';
import Login from "./Login";
import Signup from "./Signup";
import Contact from "./Contact";
import About from "./About";
import ChatBot from './ChatBot';

export {
Home,
Login,
Signup,
Contact,
About,
ChatBot,
}
```

Modular Design Notes

- Each function is independent, promoting reusability and maintainability.
- The system follows a data-driven approach, ensuring personalization and real-time adaptability.

Scope and Limitation of the Project

This section will clearly define the boundaries and objectives of the project, as well as acknowledge the inherent limitations.

a) Scope

- **Objectives:**
    - Develop a user-friendly and accessible AI-powered system that can provide personalized mental health support.
    - Utilize machine learning algorithms to analyze user data and predict potential mental health concerns.
    - Generate personalized intervention recommendations based on user data and model predictions.
    - Provide users with access to relevant resources and support.

- Ensure user privacy and data security throughout the system.

- **Target Users:**
  - Define the target user group (e.g., general population, specific age groups, individuals with specific mental health conditions).

- **System Functionality:**
  - Clearly define the core functionalities of the PMHA system, such as:
    - Data collection and processing.
    - Model training and prediction.
    - Intervention recommendation generation.
    - User interface and interaction.
    - Data storage and management.

b) Limitations

- **Data Limitations:**
  - **Data Quality:** Potential issues with data quality, such as missing values, inconsistencies, and biases in the data.
  - **Data Quantity:** Limited availability of high-quality and representative datasets for training and evaluating the system.
  - **Data Privacy and Security:** Challenges in ensuring user privacy and data security while collecting and utilizing user data.

- **Algorithmic Limitations:**
  - **Bias in AI Algorithms:** Potential for bias in machine learning models due to biases in the training data or the algorithms themselves.
  - **Model Accuracy:** Limitations in model accuracy and generalizability, especially in complex and dynamic real-world scenarios.
  - **Interpretability of Models:** Difficulty in understanding and explaining the decision-making process of complex machine learning models.

- **Ethical Considerations:**
  - **User Trust and Acceptance:** Building user trust and ensuring the ethical and responsible use of AI in mental health.
  - **Potential for Misuse:** Addressing the potential for misuse of the system, such as over-reliance on AI or the use of the system for harmful purposes.
  - **Algorithmic Fairness:** Ensuring that the system is fair and equitable for all users, regardless of their background or characteristics.
- **Technical Limitations:**
  - **Computational Resources:** Limitations in computing resources for training and deploying complex machine learning models.

- **Scalability:** Challenges in scaling the system to accommodate a large number of users and data.
- **Integration with Existing Systems:** Challenges in integrating the PMHA system with existing healthcare systems and electronic health records.

Conclusion

This section will summarize the key findings and accomplishments of the project, discuss the program outcomes, and outline potential future directions.

a) Summary of Findings

- **Summarize the key findings and accomplishments of the project:**
  - Briefly describe the development process, including data collection, model development, and system implementation.
  - Discuss the performance of the developed PMHA system, including model accuracy, user feedback, and system usability.
  - Highlight the successful integration of AI and mental health principles in the system.

b) Program Outcomes

- **Discuss the program outcomes satisfied by the work:**
  - **Improved Access to Mental Healthcare:**
    - Discuss how the PMHA system can improve access to mental healthcare by providing 24/7 support, reducing the stigma associated with seeking professional help, and offering affordable and convenient options.
  - **Personalized Mental Health Support:**
    - Emphasize the system's ability to provide personalized interventions tailored to individual user needs and preferences.
  - **Proactive Mental Health Monitoring:**
    - Discuss how the system can proactively identify potential mental health concerns and provide early interventions.
  - **Empowerment of Users:**
    - Discuss how the system can empower users to take a more active role in managing their mental health by providing them with tools and resources.
  - **Advancement of AI in Mental Health:**
    - Discuss how the project contributes to the advancement of AI in mental health by exploring innovative approaches to mental health

support and demonstrating the potential of AI to improve mental health outcomes.
    - Explore the use of more advanced deep learning models, such as transformers and graph neural networks, for improved performance.
    - Investigate the use of explainable AI (XAI) techniques to improve model interpretability and user trust.

- **Expansion of intervention options:**
    - Incorporate a wider range of interventions, such as virtual therapy sessions, social support networks, and gamified interventions.
    -
- **Addressing ethical and societal implications:**
    - Conduct further research on ethical considerations, such as data privacy, algorithmic bias, and the responsible use of AI in mental health.

- **User-centered design:**
    - Continuously gather user feedback and iterate on the system design to improve user experience and satisfaction.